Apuntes de Docker

Imágenes, contenedores y Dockerfile

FROM · LABEL · ENV · COPY · CMD · RUN · WORKDIR

Imágenes Alpine y Debian · ejemplos para copiar y pegar

Cómo usar estos apuntes

- Los bloques **azules** son `Dockerfile`; los bloques **oscuros** son comandos para la **terminal**. Se pueden copiar y pegar tal cual.
- Donde ves `jaisormar` pon **tu usuario de Docker Hub** (el nombre de la imagen siempre es `usuario/nombre:tag`).
- Si copias código desde un PDF o Word y falla, revisa las **comillas**: en un Dockerfile tienen que ser rectas (`"`), no tipográficas.
- Los recuadros verdes son lo que más probabilidad tiene de caer en el examen; los rojos, errores típicos con su solución.
- Al final tienes la sección **Errores típicos** y una **chuleta de una página**.

Índice

1. Conceptos clave 4

1.1 Virtualización: máquinas virtuales vs contenedores 4

1.2 Arquitectura de Docker 4

1.3 Vocabulario imprescindible 4

1.4 Flujo de trabajo típico 5

2. Comandos de Docker 5

2.1 Imágenes 5

2.2 Lanzar contenedores: docker run 5

2.3 Gestionar contenedores 6

2.4 Entrar en un contenedor: exec y attach 6

2.5 Crear una imagen desde un contenedor: docker commit 6

2.6 Limpieza 7

3. Dockerfile: instrucciones 7

3.1 Estructura de un Dockerfile 7

3.2 FROM 8

3.3 LABEL 8

3.4 ENV 8

3.5 WORKDIR 9

3.6 COPY (y ADD) 9

3.7 RUN 10

3.8 CMD (y ENTRYPOINT) 10

3.9 Otras instrucciones: EXPOSE, ARG, USER, VOLUME 11

3.10 Dockerfile completo con las 7 instrucciones 11

4. Imágenes Alpine vs Debian 12

4.1 Diferencias 12

4.2 Instalar paquetes en cada una 12

4.3 Cómo saber si una imagen es Alpine o Debian 12

4.4 Por qué las imágenes Alpine pesan menos (respuesta modelo) 13

5. Plantillas completas para copiar 13

5.1 Receta para resolver un ejercicio tipo examen 13

5.2 index.html de prueba 13

5.3 nginx + Alpine + nano 13

5.4 Apache (httpd) + Debian + curl 14

5.5 nginx + Debian + nano 14

5.6 Apache (httpd) + Alpine + curl 14

5.7 Comparar tamaños . . . 14
5.8 Imagen desde cero con una base mínima: Alpine y Debian . . . 14
5.9 Aplicación Python (Alpine) . . . 15

6. docker build . . . 16

6.1 Sintaxis y opciones . . . 16
6.2 Caché y .dockerignore . . . 16

7. Volúmenes . . . 17

7.1 Qué son . . . 17
7.2 Comandos . . . 17
7.3 Tipos de volumen con ejemplos . . . 17

8. Docker Hub y entrega . . . 18

8.1 Subir imágenes a Docker Hub . . . 18
8.2 Subir la práctica a GitHub . . . 18

9. Multi-stage build . . . 19

10. Buenas prácticas en Dockerfile . . . 19

11. Errores típicos y cómo arreglarlos . . . 20

12. Chuleta de una página . . . 21

1. Conceptos clave

1.1 Virtualización: máquinas virtuales vs contenedores

Virtualización = emulación por software de una infraestructura de servidores real, con un entorno de ejecución idéntico al físico. Hay dos formas: **máquinas virtuales (VM)** y **contenedores**.

	Máquina virtual	Contenedor (Docker)
Aislamiento	A nivel de hardware (hipervisor)	A nivel de sistema operativo
Sistema operativo	Cada VM lleva su SO completo	Comparten el kernel del SO host
Arranque	Minutos	Segundos
Tamaño	Varios GB	KB / MB
Consumo de recursos	Mayor	Menor
Portabilidad	Se mueve a otro host fácilmente	Se destruye y se recrea a partir de la imagen
Seguridad	Más aislado (SO propio)	Menos hermético (kernel compartido)
Necesita	Hipervisor (VirtualBox, VMware, Hyper-V)	Docker Engine

Docker es open source (2013, empresa dotCloud, escrito en Go) y usa características del kernel de Linux (**namespaces** y **cgroups**) para crear los contenedores.

1.2 Arquitectura de Docker

Docker usa una arquitectura **cliente-servidor**: el **cliente** (`docker`) manda órdenes al **daemon** (`dockerd`), que es quien crea, ejecuta y distribuye los contenedores. Cliente y daemon pueden estar en la misma máquina o en máquinas distintas. Las imágenes se descargan/suben de un **registry** (Docker Hub). **Docker Engine** = daemon + API REST + CLI.

1.3 Vocabulario imprescindible

Término	Qué es
Imagen	Plantilla **de solo lectura** formada por capas: SO base + software + configuración. Se construye con un Dockerfile.
Contenedor	Instancia **en ejecución** de una imagen. Añade una capa de lectura/escritura encima de las capas de la imagen.
Dockerfile	Fichero de texto con las instrucciones para construir una imagen (imagen base, dependencias, ficheros, comando de arranque).
Tag (etiqueta)	Versión de una imagen: `nombre:tag` (`nginx:1.26`). Si no pones tag, Docker usa `latest`.
Registry / Docker Hub	Repositorio donde se guardan y comparten imágenes. Formato de nombre: `usuario/imagen:tag`.
Volumen	Almacenamiento persistente fuera del contenedor: los datos sobreviven aunque se borre el contenedor.
Contexto	Carpeta cuyo contenido envía `docker build` al daemon (el `.` del final del comando). Aquí viven el Dockerfile y los ficheros a copiar.

1.4 Flujo de trabajo típico

Esquema

```
Dockerfile --build--> IMAGEN --run--> CONTENEDOR
                        ^  |               |
                  pull  |  | push          | commit
                        |  v               v
                     Docker Hub        (nueva IMAGEN)

Contenedor:  run -> stop / start / restart -> rm
```

Prueba de que Docker funciona: `docker run hello-world`. Por detrás: el cliente contacta con el daemon → el daemon descarga la imagen `hello-world` de Docker Hub → crea un contenedor y ejecuta el programa → redirige la salida al cliente → la terminal la muestra.

2. Comandos de Docker

> **Truco.** En muchas máquinas hay que anteponer `sudo` a todos los comandos `docker` (salvo que tu usuario esté en el grupo `docker`). En estos apuntes se omite.

2.1 Imágenes

Terminal

```
docker search nginx                          # buscar en Docker Hub
docker pull nginx:1.27-alpine                # descargar (sin tag = latest)
docker images                                # listar imágenes locales
docker image ls                              # igual que el anterior
docker image ls -a                           # todas (también las intermedias)
docker image ls -q                           # solo los IDs
docker image ls --filter "reference=*web*"   # filtrar por nombre
docker image inspect nginx:1.27-alpine       # detalle en JSON (capas, puertos, env...)
docker image history nginx:1.27-alpine       # capas y la instrucción que creó cada una
docker tag nginx:1.27-alpine jaisormar/mi-nginx:1.0   # darle otro nombre/tag
docker rmi jaisormar/mi-nginx:1.0            # borrar imagen (por nombre o por ID)
docker rmi -f IMAGEN1 IMAGEN2                # borrar varias / forzar
docker image prune                           # borrar imágenes huérfanas (<none>)
```

> **Ojo.** `docker rmi` falla si hay un contenedor (aunque esté parado) que usa esa imagen: primero `docker rm -f CONTENEDOR`.

2.2 Lanzar contenedores: docker run

Sintaxis: `docker run [OPCIONES] IMAGEN[:TAG] [COMANDO] [ARGUMENTOS]`. El comando, si lo pones, va **siempre al final** (detrás del nombre de la imagen) y sustituye al `CMD` de la imagen.

Opción	Qué hace
`-d`	**Detached**: segundo plano, deja libre el prompt. Para servicios (nginx, httpd...).
`-it`	**Interactivo** + terminal. Para entrar en una shell (`sh`, `bash`).
`--name X`	Pone nombre al contenedor. Si no, Docker inventa uno. No se puede repetir un nombre.
`--rm`	Borra el contenedor al terminar (con **dos** guiones).
`-p host:cont`	Publica un puerto: `-p 8081:80` → `localhost:8081` llega al puerto 80 del contenedor.
`-P`	Publica todos los puertos `EXPOSE` de la imagen en puertos aleatorios del host.
`-e VAR=valor`	Define una variable de entorno (se puede repetir).
`-v origen:destino`	Monta un volumen o una carpeta del host (ver sección 7).

```
Terminal
docker run hello-world
docker run -d --name web1 -p 8081:80 nginx:1.27-alpine
docker run -d --name web2 -p 127.0.0.1:8082:80 httpd:2.4.62
docker run -it --rm alpine sh
docker run -it --rm debian bash
docker run --rm alpine ifconfig
docker run --rm -e APP=MiApp -e OTRA=Hola alpine env
docker run -d -p 2000-3000:80 nginx:1.27-alpine
docker run -d -P nginx:1.27-alpine
```

Comentarios: `127.0.0.1:8082:80` solo escucha en localhost; `2000-3000:80` usa un puerto libre de ese rango; `-P` mira `docker ps` para ver el puerto asignado. Para salir de una shell: `exit`. Para salir sin parar el contenedor: **Ctrl+P y luego Ctrl+Q**.

> **Para el examen.** El puerto interno de nginx y de Apache es **siempre el 80**. `-p 8081:8081` es un error típico: lo correcto es `-p 8081:80` (puerto del host : puerto del contenedor).

2.3 Gestionar contenedores

```
Terminal
docker ps                       # contenedores en ejecución
docker ps -a                    # todos, también los parados
docker ps -aq                   # solo los IDs de todos
docker stop web1                # parar (nombre o ID)
docker start web1               # arrancar uno parado
docker restart web1
docker kill web1                # parar a la fuerza
docker pause web1
docker unpause web1
docker logs web1                # ver la salida del contenedor
docker logs -f web1             # seguir los logs en vivo (Ctrl+C para salir)
docker inspect web1             # info completa en JSON
docker rename web1 web-nuevo
docker rm web1                  # borrar (tiene que estar parado)
docker rm -f web1               # borrar aunque esté en marcha
```

Estados que verás en `docker ps -a`: `Up` (en marcha), `Exited (0)` (terminó bien), `Exited (1)`/`(127)` (terminó con error), `Created`.

2.4 Entrar en un contenedor: exec y attach

```
Terminal
docker exec web1 ls /usr/share/nginx/html     # UN comando dentro de un contenedor en marcha
docker exec web1 nano --version               # comprobar que un paquete está instalado
docker exec -it web1 sh                       # abrir shell (imágenes Alpine: sh)
docker exec -it web2 bash                     # abrir shell (imágenes Debian: bash)
docker attach web1                            # engancharse al proceso principal
docker cp index.html web1:/usr/share/nginx/html/   # copiar un fichero al contenedor
```

- `docker exec` ejecuta un comando **extra**; el contenedor tiene que estar en marcha. Al salir de la shell, el contenedor sigue vivo.
- `docker attach` te engancha al proceso **principal** (PID 1). Si sales con `exit`, puedes parar el contenedor. Se usa sobre todo para depurar.

2.5 Crear una imagen desde un contenedor: docker commit

Guarda el estado actual de un contenedor (incluidos los cambios hechos a mano) como una imagen nueva. Es lo menos recomendable (no es reproducible); lo normal es usar un Dockerfile.

Terminal

```
docker run --name prueba alpine apk add --no-cache nano
docker commit prueba mi-alpine:v1
docker run --rm mi-alpine:v1 nano --version
```

2.6 Limpieza

Terminal

```
docker rm -f $(docker ps -aq)    # borrar TODOS los contenedores
docker container prune           # borrar los contenedores parados
docker image prune               # borrar imágenes huérfanas
docker volume prune              # borrar volúmenes sin usar
docker system prune              # limpia todo lo parado / sin usar
```

3. Dockerfile: instrucciones

3.1 Estructura de un Dockerfile

- Fichero de texto llamado exactamente `Dockerfile` (sin extensión), dentro de una carpeta que es el **contexto** de construcción.
- Sintaxis: `# comentario` o `INSTRUCCIÓN argumentos`. Por convención las instrucciones van en MAYÚSCULAS.
- Se ejecuta **de arriba abajo**; cada instrucción crea una **capa** (que se guarda en caché).
- `FROM` es obligatoria y tiene que ser la primera instrucción.

Dockerfile (esqueleto con las 7 instrucciones)

```
# 1) Imagen base (obligatorio y primero)
FROM imagen:tag
# 2) Metadatos
LABEL maintainer="tu_nombre" version="1.0"
# 3) Variables de entorno
ENV VARIABLE=valor
# 4) Directorio de trabajo
WORKDIR /ruta
# 5) Ejecutar comandos al construir (instalar paquetes...)
RUN comando
# 6) Copiar ficheros del contexto a la imagen
COPY origen destino
# 7) Comando por defecto al arrancar el contenedor
CMD ["ejecutable", "parametro"]
```

Instrucción	Para qué sirve	Cuándo actúa
`FROM`	Imagen base de la que partes	Construcción
`LABEL`	Añade metadatos (autor, versión...)	Construcción (queda en la imagen)
`ENV`	Define variables de entorno	Construcción **y** ejecución
`WORKDIR`	Directorio de trabajo para lo que viene después	Construcción y ejecución
`COPY`	Copia ficheros del contexto a la imagen	Construcción
`RUN`	Ejecuta un comando y guarda el resultado en una capa	Construcción
`CMD`	Comando por defecto al arrancar el contenedor	Ejecución

Error típico. En un Dockerfile los comentarios van **en su propia línea**. Si pones `# comentario` al final de una instrucción, Docker lo trata como parte de los argumentos y puede fallar.

3.2 FROM

Indica la imagen base. Sintaxis: `FROM imagen[:tag]` (o `FROM imagen:tag AS nombre` en multi-stage). Es probablemente la instrucción más importante: de ella depende el sistema (Alpine o Debian) y, por tanto, **cómo se instalan los paquetes**. Pon siempre un tag concreto: `latest` cambia con el tiempo y puede romper tu build.

Ejemplos de FROM (elige UNO por Dockerfile)

```
# Ejemplos de FROM (en cada Dockerfile va UNO solo)
FROM alpine:3.20
FROM debian:bookworm-slim
FROM ubuntu:22.04
# nginx sobre Alpine
FROM nginx:1.27-alpine
# nginx sobre Debian (sin sufijo = Debian)
FROM nginx:1.26
# Apache sobre Debian
FROM httpd:2.4.62
# Apache sobre Alpine
FROM httpd:2.4-alpine
FROM python:alpine
```

3.3 LABEL

Añade metadatos a la imagen como pares `clave=valor`. Si el valor tiene espacios, va entre comillas dobles.

Dockerfile

```
FROM alpine:3.20
LABEL maintainer="Tu Nombre"
LABEL version="1.0"
LABEL description="Servidor web con nginx"
```

Varias etiquetas en una sola línea

```
LABEL maintainer="Tu Nombre" version="1.0" description="Servidor web con nginx"
```

Terminal

```
docker image inspect --format '{{json .Config.Labels}}' jaisormar/web-nginx:1.0
docker build --label "Author=antoni" -t demo:1 .
```

La segunda línea añade una etiqueta al construir (mismo efecto que `LABEL` dentro del Dockerfile).

3.4 ENV

Define variables de entorno que se pueden usar en las instrucciones **posteriores** del Dockerfile y que además quedan disponibles **dentro del contenedor** al ejecutarlo. Dos formas: `ENV clave=valor` (admite varias a la vez) y `ENV clave valor` (solo una).

Dockerfile

```
FROM alpine:3.20
ENV APP_NAME=MiApp
ENV AUTOR="Tu Nombre" ENTORNO=produccion
ENV miVariable 2
RUN echo "App: $APP_NAME - Entorno: $ENTORNO"
CMD ["sh", "-c", "echo Hola desde $APP_NAME, autor $AUTOR"]
```

Terminal

```
docker build -t demo-env .
docker run --rm demo-env
docker run --rm demo-env env
docker run --rm -e APP_NAME=OtraApp demo-env
```

`-e` en `docker run` **sobrescribe** el valor puesto con `ENV`. Con `docker run --rm IMAGEN env` ves todas las variables.

3.5 WORKDIR

Establece el directorio de trabajo para las instrucciones `RUN`, `CMD`, `ENTRYPOINT`, `COPY` y `ADD` que vengan **después**. Si la carpeta no existe, **se crea**. Las rutas relativas se resuelven desde él. Puede usarse varias veces (una ruta relativa se suma a la anterior).

Dockerfile

```
FROM alpine:3.20
WORKDIR /app
COPY script.sh .
RUN pwd
CMD ["sh", "script.sh"]
```

`RUN pwd` mostraría `/app` y `COPY script.sh .` copia el fichero a `/app/script.sh`. Para servidores web se suele fijar la carpeta web:

Dockerfile

```
FROM nginx:1.27-alpine
WORKDIR /usr/share/nginx/html
COPY index.html .
```

3.6 COPY (y ADD)

Copia ficheros o carpetas **del contexto** (carpeta donde está el Dockerfile) a la imagen. Formato: `COPY [--chown=usuario:grupo] origen... destino`. Si algún nombre tiene espacios, usa la forma JSON: `COPY ["mi fichero.txt", "/destino/"]`.

Ejemplos de COPY (sueltos, no es un Dockerfile completo)

```
# un fichero a una carpeta (el destino acaba en /)
COPY index.html /usr/share/nginx/html/
# un fichero con otro nombre
COPY index.html /usr/share/nginx/html/index.html
# varios ficheros (el destino debe ser una carpeta acabada en /)
COPY index.html estilo.css /usr/share/nginx/html/
# con comodines
COPY *.html /usr/share/nginx/html/
# una carpeta: se copia su CONTENIDO
COPY web/ /usr/share/nginx/html/
# ruta relativa al WORKDIR
COPY script.py .
# cambiando el propietario (solo imágenes Linux)
COPY --chown=nginx:nginx index.html /usr/share/nginx/html/
# desde otra etapa (multi-stage)
COPY --from=builder /app/server /usr/local/bin/server
```

- El origen tiene que estar **dentro del contexto**: `COPY ../algo /destino` **no** está permitido.
- El destino es una ruta absoluta o relativa al `WORKDIR`. Si no existe, se crea con todas sus carpetas.
- Si hay varios orígenes (o comodín), el destino **debe** acabar en `/`. Si no acaba en `/`, se trata como un fichero.
- `ADD` es como `COPY` pero además admite URLs y descomprime `.tar`. Se recomienda usar `COPY` salvo que lo necesites.

Servidor	Carpeta donde va el index.html	CMD de la imagen oficial
nginx (imagen oficial)	`/usr/share/nginx/html/`	`nginx -g "daemon off;"`
Apache httpd (imagen oficial)	`/usr/local/apache2/htdocs/`	`httpd-foreground`
nginx instalado a mano en Debian/Ubuntu	`/var/www/html/`	lo pones tú en el `CMD`

Para el examen. Error típico: copiar a la ruta de Apache en una imagen de nginx (o al revés). **nginx →** `/usr/share/nginx/html/`; **Apache →** `/usr/local/apache2/htdocs/`.

3.7 RUN

Ejecuta comandos **durante la construcción** de la imagen y guarda el resultado en una capa nueva (instalar paquetes, crear carpetas...). Admite dos formatos: **shell** (`RUN comando`, se ejecuta con `/bin/sh -c`) y **exec** (`RUN ["ejecutable", "param1"]`). Se encadenan comandos con `&&` y se parte la línea con `\`.

Dockerfile

```
FROM debian:bookworm-slim
RUN echo "hola desde RUN"
RUN ["echo", "forma exec"]
RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*
RUN mkdir -p /datos && touch /datos/fichero.txt
```

RUN en varias líneas (una sola capa)

```
FROM debian:bookworm-slim
RUN apt-get update && \
    apt-get install -y nano curl && \
    rm -rf /var/lib/apt/lists/*
```

Truco. Agrupa los comandos relacionados en **un solo RUN**: cada instrucción es una capa y menos capas = imagen más pequeña y limpia. `apt-get update` y `apt-get install` siempre en el mismo `RUN`.

3.8 CMD (y ENTRYPOINT)

`CMD` define el **comando por defecto al arrancar el contenedor** (`docker run`). Si en `docker run` pones otro comando al final, **sustituye** al `CMD`. Solo cuenta el **último** `CMD` del Dockerfile. Tiene tres formatos:

- `CMD ["ejecutable", "param1", "param2"]` → **exec** (el preferido, JSON con comillas dobles).
- `CMD comando param1 param2` → **shell** (se ejecuta con `/bin/sh -c`).
- `CMD ["param1", "param2"]` → parámetros por defecto para un `ENTRYPOINT`.

Ejemplos de CMD (en un Dockerfile solo se usa uno)

```
CMD ["nginx", "-g", "daemon off;"]
CMD ["httpd-foreground"]
CMD ["python", "script.py"]
CMD ["sh", "-c", "echo Hola $NOMBRE"]
CMD echo "forma shell"
```

	RUN	CMD
Cuándo se ejecuta	Al **construir** la imagen (`docker build`)	Al **arrancar** el contenedor (`docker run`)
Cuántos puede haber	Todos los que quieras (una capa cada uno)	Solo vale el último
Se puede sobrescribir	No	Sí: `docker run IMAGEN otro_comando`
Típico	Instalar paquetes	Arrancar el servidor / la aplicación

- El proceso del `CMD` debe quedarse **en primer plano**; si termina, el contenedor se para. Por eso nginx lleva `daemon off;`.
- La forma correcta es `"daemon off;"` (con espacio). En algunas diapositivas aparece `daemon:off;` por una errata.
- En forma **exec** no hay shell, así que `$VARIABLE` **no se expande**. Si la necesitas: `CMD ["sh", "-c", "echo $VARIABLE"]`.

ENTRYPOINT + CMD

`ENTRYPOINT` fija el ejecutable que siempre se lanza; `CMD` aporta los parámetros **por defecto**, que se pueden cambiar al hacer `docker run`. Si no hay `ENTRYPOINT`, lo que se ejecuta es `/bin/sh -c`.

Dockerfile

```
FROM alpine:3.20
ENTRYPOINT ["echo"]
CMD ["Hola mundo"]
```

Terminal

```
docker build -t demo-entry .
docker run --rm demo-entry
docker run --rm demo-entry Adios
docker run --rm --entrypoint date demo-entry
```

Resultados: `Hola mundo` · `Adios` (los argumentos sustituyen al `CMD`) · la fecha (`--entrypoint` anula el `ENTRYPOINT` del Dockerfile).

3.9 Otras instrucciones: EXPOSE, ARG, USER, VOLUME

Dockerfile

```
FROM debian:bookworm-slim
ARG VERSION=1.0
LABEL version=$VERSION
RUN useradd -m antoni
USER antoni
EXPOSE 80
VOLUME /var/log
```

- `EXPOSE 80` **documenta** que el contenedor escucha en ese puerto, pero **no lo publica**: hay que usar `-p` en `docker run`. Admite protocolo: `EXPOSE 53/udp` (por defecto TCP).
- `ARG` define variables **solo durante la construcción**: `docker build --build-arg VERSION=2.0 -t demo:2.0 .` (a diferencia de `ENV`, no existen en el contenedor).
- `USER` fija el usuario con el que se ejecutan los `RUN`, `CMD` y `ENTRYPOINT` siguientes. Crear usuario: en Alpine `adduser -D antoni`, en Debian `useradd -m antoni`.
- `VOLUME /ruta` crea un punto de montaje (volumen anónimo) en esa ruta.

3.10 Dockerfile completo con las 7 instrucciones

Versión Alpine (nginx)

```
FROM nginx:1.27-alpine
LABEL maintainer="Tu Nombre" version="1.0"
ENV AUTOR="Tu Nombre"
WORKDIR /usr/share/nginx/html
RUN apk add --no-cache nano curl
COPY index.html .
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Versión Debian (Apache)

```
FROM httpd:2.4.62
LABEL maintainer="Tu Nombre" version="1.0"
ENV AUTOR="Tu Nombre"
WORKDIR /usr/local/apache2/htdocs
RUN apt-get update && \
    apt-get install -y nano curl && \
    rm -rf /var/lib/apt/lists/*
COPY index.html .
EXPOSE 80
CMD ["httpd-foreground"]
```

Línea a línea: base → metadatos → variable → carpeta de trabajo (la web) → instalar paquetes → copiar la página (a la carpeta de trabajo) → documentar puerto → arrancar el servidor en primer plano.

4. Imágenes Alpine vs Debian

4.1 Diferencias

	Alpine	Debian (y Ubuntu)
Tamaño	Base de ~8 MB; imágenes finales de pocas decenas de MB	Base de ~75-120 MB; imágenes finales de 150-200 MB
Gestor de paquetes	`apk`	`apt-get` (o `apt`)
Instalar	`apk add --no-cache paquete`	`apt-get update && apt-get install -y paquete`
Limpiar caché	`--no-cache` ya lo evita	`rm -rf /var/lib/apt/lists/*` en el mismo `RUN`
Shell	`sh` (BusyBox/ash); **no hay bash** por defecto	`bash` y `sh`
Librería C	musl	glibc
Utilidades	BusyBox (lo mínimo)	GNU completas, muchas más bibliotecas
Cómo reconocerla	El tag lleva `alpine`: `nginx:1.27-alpine`	El resto de tags: `nginx:1.26`, `httpd:2.4.62`, `-slim`, `bookworm`

4.2 Instalar paquetes en cada una

Alpine: apk add --no-cache

```
FROM nginx:1.27-alpine
RUN apk add --no-cache nano
```

Debian: apt-get update + install -y + limpiar

```
FROM nginx:1.26
RUN apt-get update && \
    apt-get install -y nano && \
    rm -rf /var/lib/apt/lists/*
```

- **Alpine**: `apk add --no-cache` (sin `update` previo). Varios paquetes: `apk add --no-cache nano curl`.
- **Debian/Ubuntu**: `apt-get update` antes del `install`, y `-y` para que no se quede esperando confirmación (en un build no hay nadie para contestar).
- Borrar `/var/lib/apt/lists/*` no es obligatorio, pero reduce el tamaño; hazlo **en el mismo** `RUN` (si lo haces en otro, la capa anterior ya guardó los ficheros).

> **Para el examen.** `apt-get: not found` → la imagen es **Alpine**, usa `apk`. `apk: not found` → la imagen es **Debian**, usa `apt-get`.

4.3 Cómo saber si una imagen es Alpine o Debian

Terminal

```
docker run --rm nginx:1.27-alpine cat /etc/os-release
docker run --rm nginx:1.26 cat /etc/os-release
docker run --rm httpd:2.4.62 cat /etc/os-release
docker run --rm httpd:2.4-alpine cat /etc/os-release
```

En la salida, `NAME="Alpine Linux"` o `PRETTY_NAME="Debian GNU/Linux 12 (bookworm)"`.

4.4 Por qué las imágenes Alpine pesan menos (respuesta modelo)

«La imagen más pequeña es la basada en Alpine. Alpine es una distribución Linux mínima pensada para contenedores: usa la librería musl en lugar de glibc y BusyBox en lugar de las herramientas GNU, y solo incluye lo imprescindible. Debian incluye muchas más bibliotecas y utilidades, y además `apt-get` deja la lista de paquetes en la imagen si no se borra.»

5. Plantillas completas para copiar

5.1 Receta para resolver un ejercicio tipo examen

- **Lee el enunciado y apunta**: imagen base y tag, paquete a instalar, nombre de la imagen (`usuario/nombre:tag`), nombre del contenedor y puerto.
- **¿Alpine o Debian?** Si el tag lleva `alpine` → `apk add --no-cache`. Si no → `apt-get update && apt-get install -y`.
- **Ruta de la web**: nginx → `/usr/share/nginx/html/`, Apache → `/usr/local/apache2/htdocs/`.
- **Carpeta propia** con el `Dockerfile` y el `index.html`.
- `docker build -t usuario/nombre:tag .` (¡con el punto del contexto, separado por un espacio!).
- `docker run -d --name NOMBRE -p PUERTO:80 usuario/nombre:tag`.
- **Comprueba**: `curl http://localhost:PUERTO` y `docker exec NOMBRE paquete --version`.
- Si lo piden: `docker login` + `docker push`, y subir la carpeta a GitHub.

Terminal

```
mkdir -p ~/docker-imatges/ex1 && cd ~/docker-imatges/ex1
vim index.html
vim Dockerfile
docker build -t jaisormar/web-nginx:1.0 .
docker run -d --name web1 -p 8081:80 jaisormar/web-nginx:1.0
curl http://localhost:8081
docker exec web1 nano --version
```

5.2 index.html de prueba

index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Funciona</title>
</head>
<body>
  <h1>¡Funciona!</h1>
</body>
</html>
```

5.3 nginx + Alpine + nano

Dockerfile

```
FROM nginx:1.27-alpine
RUN apk add --no-cache nano
COPY index.html /usr/share/nginx/html/index.html
```

Terminal

```
docker build -t jaisormar/web-nginx:1.0 .
docker run -d --name web1 -p 8081:80 jaisormar/web-nginx:1.0
curl http://localhost:8081
docker exec web1 nano --version
```

5.4 Apache (httpd) + Debian + curl

Dockerfile

```
FROM httpd:2.4.62
RUN apt-get update && \
    apt-get install -y curl && \
    rm -rf /var/lib/apt/lists/*
COPY index.html /usr/local/apache2/htdocs/index.html
```

Terminal

```
docker build -t jaisormar/web-apache:1.0 .
docker run -d --name web2 -p 8082:80 jaisormar/web-apache:1.0
curl http://localhost:8082
docker exec web2 curl --version
```

5.5 nginx + Debian + nano

Dockerfile

```
FROM nginx:1.26
RUN apt-get update && \
    apt-get install -y nano && \
    rm -rf /var/lib/apt/lists/*
COPY index.html /usr/share/nginx/html/index.html
```

Terminal

```
docker build -t jaisormar/web-nginx:2.0 .
docker run -d --name web3 -p 8083:80 jaisormar/web-nginx:2.0
curl http://localhost:8083
docker exec web3 nano --version
```

5.6 Apache (httpd) + Alpine + curl

Dockerfile

```
FROM httpd:2.4-alpine
RUN apk add --no-cache curl
COPY index.html /usr/local/apache2/htdocs/index.html
```

Terminal

```
docker build -t jaisormar/web-apache:2.0 .
docker run -d --name web4 -p 8084:80 jaisormar/web-apache:2.0
curl http://localhost:8084
docker exec web4 curl --version
```

5.7 Comparar tamaños

Terminal

```
docker images
docker images "jaisormar/*"
docker image ls --filter "reference=jaisormar/*"
```

Las basadas en Alpine (`web-nginx:1.0`, `web-apache:2.0`) ocupan unas pocas decenas de MB; las basadas en Debian (`web-apache:1.0`, `web-nginx:2.0`) rondan los 150-200 MB. Explicación en la sección 4.4.

5.8 Imagen desde cero con una base mínima: Alpine y Debian

Mismo objetivo con las dos bases: instalar `curl` y `nano`, definir una variable y arrancar un comando.

Alpine

```
FROM alpine:3.20
LABEL maintainer="Tu Nombre"
ENV NOMBRE=Mundo
WORKDIR /app
RUN apk add --no-cache curl nano
CMD ["sh", "-c", "echo Hola $NOMBRE"]
```

Debian

```
FROM debian:bookworm-slim
LABEL maintainer="Tu Nombre"
ENV NOMBRE=Mundo
WORKDIR /app
RUN apt-get update && \
    apt-get install -y curl nano && \
    rm -rf /var/lib/apt/lists/*
CMD ["sh", "-c", "echo Hola $NOMBRE"]
```

Terminal

```
docker build -t demo:alpine .
docker run --rm demo:alpine
docker run --rm -e NOMBRE=Pepe demo:alpine
docker run --rm demo:alpine curl --version
```

Servidor web instalado a mano sobre Debian (la web va en `/var/www/html/`):

Dockerfile

```
FROM debian:bookworm-slim
RUN apt-get update && \
    apt-get install -y nginx && \
    rm -rf /var/lib/apt/lists/*
COPY index.html /var/www/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Terminal

```
docker build -t jaisormar/nginx-debian:1.0 .
docker run -d --name web5 -p 8085:80 jaisormar/nginx-debian:1.0
curl http://localhost:8085
```

5.9 Aplicación Python (Alpine)

requirements.txt

```
requests
```

script.py

```
import requests
r = requests.get("https://example.com")
print(r.status_code)
```

Dockerfile

```
FROM python:alpine
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY script.py .
CMD ["python", "script.py"]
```

Terminal

```
docker build -t app-python:v1 .
docker run --rm app-python:v1
```

Error típico. Copia `requirements.txt` **antes** del `pip install`: si no, dentro de la imagen no existe y falla con «Could not open requirements file». Copiarlo antes del script también aprovecha la caché (si solo cambias `script.py`, no se reinstalan las dependencias). Con Debian: `FROM python:3.12-slim`.

6. docker build

6.1 Sintaxis y opciones

`docker build [OPCIONES] RUTA | URL | -`. Parte de un **Dockerfile** y de un **contexto** (la ruta). El `.` del final es el contexto = carpeta actual.

Opción	Qué hace
`-t nombre:tag`	Etiqueta la imagen. Sin `-t` queda una imagen huérfana (`<none>`). Se puede repetir para poner varios tags.
`-f fichero`	Usa un Dockerfile con otro nombre (`-f Dockerfile.alpine`).
`--label k=v`	Añade metadatos (igual que `LABEL`).
`--build-arg K=v`	Da valor a un `ARG` del Dockerfile.
`--no-cache`	Ignora la caché y rehace todas las capas.
`--pull`	Fuerza descargar la versión más reciente de la imagen del `FROM`.
`-q` / `--quiet`	Solo muestra el ID de la imagen creada.
`--target etapa`	En multi-stage, construye solo hasta esa etapa.

Terminal

```
docker build -t jaisormar/web-nginx:1.0 .
docker build -t jaisormar/web-nginx:1.0 -t jaisormar/web-nginx:latest .
docker build -f Dockerfile.alpine -t demo:alpine .
docker build --no-cache -t demo:1 .
docker build --pull -t demo:1 .
docker build -q -t demo:1 .
docker build --build-arg VERSION=2.0 -t demo:2.0 .
docker build --target builder -t demo:builder .
docker build -t demo:1 /ruta/a/otra/carpeta
```

Truco. El aviso «DEPRECATED: The legacy builder is deprecated...» **no es un error**: la imagen se construye igual. Para quitarlo, instala el paquete `docker-buildx` de tu distribución.

Error típico. Tag con punto de más: `-t usuario/nombre:1.0.` crea la imagen con el tag `1.0.` (con punto). Escribe `-t usuario/nombre:1.0 .` (el punto del contexto va **separado por espacio**). Lo mismo con `docker exec web2 curl --version` (sin punto detrás de `--version`).

6.2 Caché y .dockerignore

- Docker guarda en caché cada capa («Using cache»). Si una instrucción cambia, **se rehacen esa y todas las siguientes**. Por eso se ponen arriba las instrucciones que cambian poco (instalar dependencias) y abajo las que cambian mucho (copiar tu código).
- El archivo `.dockerignore` (en el contexto) lista lo que `docker build` no debe enviar ni copiar a la imagen.

.dockerignore

```
.git/
node_modules/
dist/
*.log
```

7. Volúmenes

7.1 Qué son

Carpetas gestionadas por Docker (en `/var/lib/docker/volumes/NOMBRE/_data`) que **sobreviven al ciclo de vida del contenedor**: se puede parar, cambiar o borrar el contenedor sin perder los datos. Se pueden **compartir entre contenedores**.

- Docker **nunca borra volúmenes automáticamente** al eliminar un contenedor.
- No se puede borrar un volumen que esté asociado a un contenedor.
- Los volúmenes **anónimos** se quedan «parasitando» hasta que los borras tú con `docker volume rm`/`prune`.

7.2 Comandos

Terminal

```
docker volume create datos
docker volume ls
docker volume inspect datos
docker volume rm datos
docker volume prune
```

7.3 Tipos de volumen con ejemplos

Tipo	Cómo se crea	Características
Anónimo	`-v /data`	Docker le pone un nombre aleatorio. Se elimina solo con el último contenedor si se usó `--rm`; si no, queda huérfano.
Con nombre (named)	`-v datos:/data`	Independiente de cualquier contenedor. Persiste siempre; se borra a mano.
Montaje (bind mount)	`-v /ruta/host:/ruta/cont`	Monta una carpeta o fichero **del host** dentro del contenedor.

Anónimo, con nombre y comprobación de persistencia

```
docker run -d --name a1 -v /data alpine sleep 3600
docker run -d --name n1 -v datos:/data alpine sleep 3600
docker run --rm -v datos:/data alpine sh -c "echo hola > /data/a.txt"
docker run --rm -v datos:/data alpine cat /data/a.txt
```

Bind mount (carpeta/fichero del host)

```
docker run -d --name web -p 8080:80 -v "$(pwd)":/usr/share/nginx/html nginx:1.27-alpine
docker run -d --name web6 -p 8086:80 -v "$(pwd)":/usr/local/apache2/htdocs/ httpd:2.4.62
docker run --rm -v /etc/hostname:/root/parent_name alpine cat /root/parent_name
docker run -d --name web7 -p 8087:80 -v "$(pwd)":/usr/share/nginx/html:ro nginx:1.27-alpine
```

El `:ro` del último ejemplo monta en **solo lectura**. Con un bind mount puedes editar el `index.html` en tu máquina y verlo al momento sin reconstruir la imagen.

8. Docker Hub y entrega

8.1 Subir imágenes a Docker Hub

```
Terminal
docker login
docker push jaisormar/web-nginx:1.0
docker push jaisormar/web-apache:1.0
docker push jaisormar/web-nginx:2.0
docker push jaisormar/web-apache:2.0
docker pull jaisormar/web-nginx:1.0
docker logout
```

Comprueba en `https://hub.docker.com/u/TU_USUARIO` que aparecen con su tag. Si la imagen se creó **sin el usuario delante**, el push falla con `denied: requested access to the resource is denied`; se arregla reetiquetando:

```
Terminal
docker tag web-nginx:1.0 jaisormar/web-nginx:1.0
docker push jaisormar/web-nginx:1.0
```

> **Para el examen.** El nombre completo es `usuario/imagen:tag`. Sin `docker login` o sin el `usuario/` delante, el push da «denied».

8.2 Subir la práctica a GitHub

Crea antes el repositorio **vacío** en GitHub (por ejemplo `practica-dockerfile-1`). Estructura: `ex1/`…`ex4/` con su `Dockerfile` e `index.html`, y un `README.md` con tu nombre y la respuesta de la pregunta de los tamaños.

```
Terminal
cd ~/docker-imatges
git init
git add .
git commit -m "Práctica Dockerfile 1"
git branch -M main
git remote add origin https://github.com/jaisormar/practica-dockerfile-1.git
git push -u origin main
```

Con la CLI de GitHub, todo de una vez: `gh repo create practica-dockerfile-1 --public --source=. --push`.

9. Multi-stage build

Permite usar **varios** `FROM` en un mismo Dockerfile. Se construye en una etapa «pesada» (con compilador, herramientas...) y se copia solo el resultado a una imagen final pequeña. **Vale la última etapa.** `COPY --from=etapa` copia ficheros de una etapa anterior (o de una imagen); `docker build --target etapa` construye solo hasta esa etapa.

Sin multi-stage: la imagen final pesa lo que golang (~800 MB)

```
FROM golang:1.22
WORKDIR /app
COPY main.go .
RUN go build -o server main.go
CMD ["./server"]
```

Con multi-stage: la imagen final pesa lo que Alpine + el binario

```
FROM golang:1.22 AS builder
WORKDIR /app
COPY main.go .
RUN CGO_ENABLED=0 go build -o server main.go

FROM alpine:3.20
WORKDIR /app
COPY --from=builder /app/server .
EXPOSE 3000
CMD ["./server"]
```

Ejemplo con Java (Maven)

```
FROM maven:3.6.2-ibmjava-8 AS builder
COPY app /app
RUN cd /app && mvn package

FROM openjdk:8-alpine
COPY --from=builder /app/target/ejecutable.jar /opt/ejecutable.jar
CMD ["java", "-jar", "/opt/ejecutable.jar"]
```

Terminal

```
docker build -t antoni/maven-stage .
docker build --target builder -t antoni/maven-stage:builder .
```

10. Buenas prácticas en Dockerfile

- **Etiqueta la imagen base** con una versión concreta (`FROM ubuntu:22.04`, no `FROM ubuntu`), para que el build no se rompa cuando `latest` cambie.
- **Fusiona comandos** `RUN` relacionados con `&&` y `\`: menos capas.
- **Limpia tras instalar** en el mismo `RUN`: `rm -rf /var/lib/apt/lists/*` (Debian/Ubuntu) o `apk add --no-cache` (Alpine).
- **Usa** `.dockerignore` para no enviar `.git/`, `node_modules/`, etc.
- **Prefiere** `COPY` **a** `ADD`, y la forma **exec** en `CMD` (`["a", "b"]`).
- **Ordena** las instrucciones de menos a más cambiantes para aprovechar la caché.
- **Añade metadatos** con `LABEL` (autor, versión, descripción).
- **Imágenes mínimas** (Alpine, `-slim`) y, si compilas, **multi-stage**.
- Un contenedor = **un proceso principal** en primer plano.

Mal

```
FROM ubuntu
ADD . /app
RUN apt-get update
RUN apt-get install -y nodejs
RUN cd /app && npm install
CMD npm start
```

Mejor

```
FROM ubuntu:22.04
RUN apt-get update && \
    apt-get install -y nodejs && \
    rm -rf /var/lib/apt/lists/*
COPY . /app
WORKDIR /app
RUN npm install
CMD ["npm", "start"]
```

11. Errores típicos y cómo arreglarlos

Síntoma	Causa	Solución
`apt-get: not found`	La imagen base es Alpine	`RUN apk add --no-cache paquete`
`apk: not found`	La imagen base es Debian	`RUN apt-get update && apt-get install -y paquete`
`Could not open requirements file`	No copiaste `requirements.txt` antes del `pip install`	`COPY requirements.txt .` antes del `RUN pip install`
`COPY failed: file not found in build context`	El fichero no está en la carpeta del Dockerfile o está mal escrito	Copia el fichero al contexto; revisa el nombre. No uses `../`
La web no carga / sigue la de por defecto	Copiaste a la carpeta equivocada	nginx: `/usr/share/nginx/html/` · Apache: `/usr/local/apache2/htdocs/`
`curl: option --version.: is unknown`	Punto de más al final del comando	`docker exec web2 curl --version` (sin punto)
Imagen con tag `1.0.` (con punto)	Punto pegado al tag en `-t`	`-t usuario/img:1.0 .` · arréglalo con `docker tag` y `docker rmi`
No abre en `localhost:8081`	`-p 8081:8081`	`-p 8081:80` (host:contenedor)
`The container name "/web1" is already in use`	Ya existe un contenedor con ese nombre	`docker rm -f web1` y vuelve a lanzarlo
`port is already allocated`	Ese puerto del host está ocupado	Cambia el puerto del host (`-p 8091:80`) o para el otro contenedor
`denied: requested access to the resource is denied`	Falta `docker login` o el `usuario/` en el nombre	`docker login`; `docker tag img usuario/img:tag`
El contenedor sale al instante (`Exited (0)`)	El `CMD` termina; el proceso no queda en primer plano	Servidor en primer plano (`daemon off;`, `httpd-foreground`)
`unable to remove image ... in use`	Un contenedor usa la imagen	`docker rm -f contenedor` y luego `docker rmi`
`Cannot connect to the Docker daemon`	El servicio está parado o no tienes permisos	`sudo systemctl start docker`; usa `sudo` o el grupo `docker`
`CMD` no arranca / error de JSON	Comillas tipográficas “ ” o simples en la forma exec	Usa comillas dobles rectas `"`
Falla una instrucción con comentario al final	`#` en la misma línea	Comentarios en su propia línea

12. Chuleta de una página

Comando	Para qué
`docker build -t usuario/nombre:tag .`	Construir imagen desde el Dockerfile de la carpeta
`docker run -d --name X -p 8081:80 IMG`	Contenedor en segundo plano, puerto 8081 → 80
`docker run -it --rm alpine sh`	Shell temporal (Debian: `debian bash`)
`docker run -e VAR=valor IMG`	Variable de entorno
`docker run -v datos:/data IMG`	Volumen con nombre (bind mount: `-v "$(pwd)":/ruta`)
`docker ps` / `docker ps -a`	Contenedores en marcha / todos
`docker stop X` · `start X` · `restart X`	Parar / arrancar / reiniciar
`docker rm X` · `docker rm -f X`	Borrar contenedor (parado / forzado)
`docker logs -f X`	Ver logs en vivo
`docker exec X comando`	Ejecutar un comando en un contenedor en marcha
`docker exec -it X sh`	Entrar con shell (Debian: `bash`)
`docker images` · `docker image ls -q`	Listar imágenes / solo IDs
`docker rmi IMG`	Borrar imagen
`docker image inspect IMG` · `history IMG`	Detalle / capas de la imagen
`docker tag ORIGEN usuario/nombre:tag`	Poner otro nombre o tag
`docker login` · `docker push IMG` · `docker pull IMG`	Docker Hub
`docker commit X nombre:tag`	Imagen a partir de un contenedor
`docker volume create|ls|inspect|rm|prune`	Gestión de volúmenes
`docker system prune`	Limpiar lo parado / sin usar

Dockerfile (versión Alpine)

```
FROM imagen:tag
LABEL maintainer="Nombre" version="1.0"
ENV VARIABLE=valor
WORKDIR /ruta
RUN apk add --no-cache paquete
COPY fichero destino
CMD ["ejecutable", "parametro"]
```

En **Debian** cambia solo la línea del `RUN`: `RUN apt-get update && apt-get install -y paquete && rm -rf /var/lib/apt/lists/*`

nginx	web en `/usr/share/nginx/html/`	Alpine: `nginx:1.27-alpine` · Debian: `nginx:1.26`
Apache	web en `/usr/local/apache2/htdocs/`	Alpine: `httpd:2.4-alpine` · Debian: `httpd:2.4.62`
Puerto interno	siempre `80`	`-p PUERTO_HOST:80`